Mining quantitative temporal dependencies between interval-based streams

Amine El Ouassouli[1,2], Lionel Robinault[2], and Marian Scuturici[1]

[1] INSA de Lyon {amine.el-ouassouli,marian.scuturici}@insa-lyon.fr
[2] Foxstream {a.elouassouli, l.robinault}@foxstream.fr

Abstract. Data streams gathered from sensor systems can contain a significant amount of noise and are challenging to sequential pattern mining algorithms. A majority of existing approaches deals with such data as time point events to find *before/after* relations that induces loss of information when dealing with events lasting in time, i.e intervals. Other interval-based approaches focus on qualitative patterns and are sensitive to temporal variability. We consider than quantitative patterns maintaining lag information permit to deal better with this problem. In this work, we propose an efficient algorithm devised to extract quantitative patterns from interval streams: Interval Time Lag Discovery(ITLD). It is based on an intersection-based confidence that is automatically assessed with a statistical χ^2 test. Experimental results, on both synthetic and real-life data, show that our method is more suitable for sensor interval streams and provides more precise information in comparison with existing approaches.

1 Introduction

Extensive usage of sensors makes it possible to obtain a precise monitoring of temporal activities. In many application domains, the data produced by the monitoring raises the need to extract patterns describing hidden temporal phenomena or behaviors.

As a motivation example, we describe in Fig. 1 and Fig. 2 a sensor system composed of 4 outdoor cameras situated in an office area. These cameras are capturing motion using real time video processing. Starting from images taken by these cameras, we defined 10 "virtual" motion sensors (displayed with red polygons) corresponding to physical regions and labeled as 1-1, 1-2, ..., 4-3. Each virtual motion sensor produces a sequence of time point events of two types: *"B: Motion Begin"* and *"E: Motion End"*. We can associate these events with a change in the state of the data source. Such a sequence can be transformed to an interval-based state sequence composed of intervals $[t_b, t_e)$ signaling the activation of an environment state *"Motion in area x"*, noted *Mx* for concision. For example, Fig.3 describes an initial sequence of raw time point events produced by *4-1* and *4-3* and Fig.4 shows the corresponding interval sequences. We consider that an interval based representation provides more insight about the studied phenomenon. Our objective is to identify temporal correlations between

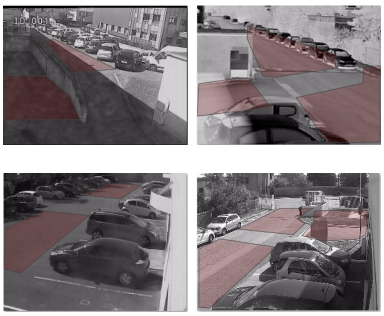


Fig. 1: Four outdoor camera views. Red contours describe motion analysis areas.

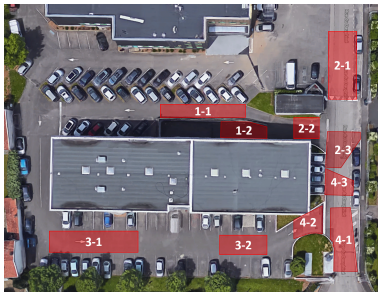


Fig. 2: Position of motion analysis areas (aerial view)

these state streams in order to describe quantitatively typical trajectories in this environment. For example, the temporal relation "M4-1 $\rightarrow$ M4-3$^{(3,2)}$" means that the activation of the state *M4-3* appears 3 time units after the activation of *M4-1*, and the state is deactivated 2 time units after the end of *M4-1*. In Fig.2 this relation is not always true, and we evaluate the confidence in this relation at 0.7.

Sequential and temporal pattern mining is a well studied problem. Most of the existing algorithms (e.g [6] [8]) focus on point-based data (events without duration) and are designed to extract qualitative patterns describing *before/after* relations. In the case of interval-based data (events with duration), using such approaches implies loss of information as they do not take into account event duration. As discussed in [1], richer relations can be expressed using interval-based models. Most of existing quantitative algorithms utilizes pattern models based on Allen' logic. This makes them sensitive to variability of data as slight changes in interval endpoints can lead to different qualitative relations. More expressive quantitative patterns, including time lag information, permit to deal better with this problem. Among the existing approaches, the majority [3] [7] [10] do not allow to process straightforwardly interval streams. Input data for these algorithms is assumed to be a transaction database of sequences identified by an ID: each transaction corresponds to a time-stamped activity. Our problem is different: sensor data are unbounded sequences with no separation between activities.

Two existing algorithms, PIVOTMiner [4] and TEDDY [9] permit to process interval streams. PIVOTMiner is a support-based algorithm while TEDDY

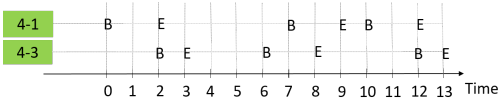


Fig. 3: Raw event streams produced by sensors *4-1* and *4-3* in Fig. 2.

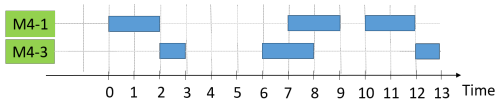


Fig. 4: State (interval) streams built from Fig. 3 raw data

considers a novel confidence measure based on intervals intersection length. In the case of sensor data the usage of the intersection between intervals allows a better evaluation of certain temporal relations. Given the example described in Fig. 2, if several pedestrians cross successively *4-1* then *4-3*, a single long interval for each analysis zone will be generated: a support-based assessment will count 1 occurrence while intersection will be impacted according to the intersection length. To validate the detected temporal relations, PIVOTMiner uses a given minimum support as a significance criterion and TEDDY considers the automatic statistical assessment on intersection length to define a significance threshold.

This work focuses on the problem of finding pairwise relations describing significant time lags between interval-based state streams. More precisely, we extend the intersection-based pattern model introduced in [9] and propose an efficient and precise time lag discovery algorithm, named **I**nterval **T**ime **L**ag **D**iscovery (ITLD), that is based on the statistical assessment of the confidence variation. We also report on experiments on both synthetic and real life data showing that ITLD is capable of discovering efficiently and precisely significant time lags between pair of state streams in comparison with the existing approaches.

The rest of this paper is organized as follows: Sec.2 surveys temporal pattern mining approaches; Sec.3 introduces necessary definitions; Sec.4 introduces the ITLD algorithm; Sec.5 reports on experiments on ILTD and provides comparison with existing algorithms and finally Sec.6 concludes this paper and discusses some perspectives.

2 Related work

Sequential and temporal pattern mining is a well studied research area designed to discover regularities among temporally ordered data. Contributions in this field can be categorized following three main criteria: data format (sequence database or streams/single sequences), time models (time points or intervals) and temporal description (qualitative or quantitative).

The pattern mining problem differs following the input data format. On the one hand, this task consists of finding regularities between transactions within a database of temporally ordered sequences. Each sequence describes an activity/behavior associated with an unique ID referring to a subject (e.g market basket data or medical records). On the other hand, pattern mining is designed to discover frequent relations within a unique sequence/stream of items without

separation between activities. In both cases, existing contributions use a user-given minimum support referring to the part of data where a relation stands. The support assessment can be based on number of transaction for sequence databases, time windows [6] or number of items ([11]). Our work is closer to the second category: items without separation between activities.

Most of the contributions dealing with streaming data or single sequences focus on time points with qualitative patterns reporting on *before/after/co-occurring* relations (e.g [6]). Quantitative patterns do not permit a time delay discrimination which is rather useful for many application domains. While some contributions integrate temporal constraints (time windows [6], gap constraint [2]), several approaches tackled quantifying time delays ([12] [11] [13]). Another category of contributions deals with interval data. As discussed in [1] this time model permits to express more complex relations (13 relations in Allen' logic). This form of patterns may suffer from expressiveness limitations as ambiguity (a pattern may lead to different relations) or completeness (not allowing to express all possible relations). We refer to [5] for a detailed analysis of expressiveness. Besides, qualitative patterns are sensitive to temporal variability as slight variation in intervals endpoints that may lead to different qualitative relations.

Quantitative patterns permit to deal with these problems as they include temporal information permitting to infer Allen' logic. Among the existing contributions [7] [3] [10] only two recent approaches are designed or can be easily adapted to interval streams [4] [9]. In [4], authors propose PIVOTMiner that uses a geometric approach consisting on the projection of each interval $[t_b, t_e)$ into a bi-dimensional plane (begin, end) where the temporal relation between two intervals is considered as a vector. This allows to perform a DBSCAN clustering after an origin transformation stage: to mine $A \rightarrow B$, all vector with A as source and B as target are moved such as all sources coincides with the space origin. Cluster centroides provides the time lag information. While designed for databases of sequences, this approach can be easily adapted to data streams as it is not endpoint sensitive (as in [10], [3]). Finally, in [9] the authors proposed a novel form of temporal relations made possible with usage of intervals.It is based on intersection of streams. In that work, a data stream is formatted as several sub-streams containing each a unique label type. The main idea is to use a χ^2 test of independence to compare the observed intersection length with the expected one under the independence hypothesis. The latter, uses the assumption of uniform distribution of B's length. The authors propose an algorithm, TEDDY, devised to discover such dependencies given a temporal constraint Δ on temporal transformation and by extension time lags. It explores the search space as a semi-lattice defined by transformation inclusion (i.e if an (α, β)-transformation sequence contains all intervals of (α', β')-transformation then (α, β) includes (α', β')) using a breadth-first strategy. As the used confidence measure is monotonic, multiple temporal dependencies can express the same temporal relation. Therefore, TEDDY integrates pruning criteria based on a dominance relationship permitting to select the most specific dependencies by

refining temporal transformation while controlling the loss in the intersection-based confidence measure.

3 Definitions and Problem statement

Data streams We define a data stream D as a sequence of timestamped data produced by a source d: $D = \{(t, v) \mid t \in \mathcal{T}, v \in \mathcal{V}\}$. $\mathcal{T}$ is an infinite set of discrete timestamps and $\mathcal{V}$ is the set of possible values given by D. We assume that a data source cannot produce more than one value at a time. In the experiment described higher, $\mathcal{V}$ is the set of possible event types $\mathcal{V} = \{B, E\}$. In Fig.3, the data stream corresponding to data source *4-1* is $\{(B, 0), (E, 2), (B, 7)...(E, 12)\}$

States and State streams A state corresponds to a data configuration of interest for the application domain. It is defined as a predicate on data produced by a data source. For example, the state *Motion in area 4-1*, noted *M4-1* in Fig.4, is defined with $M4\text{-}1(t) = (Last(4\text{-}1, t) == B)$ denoting that environment is at state *Motion in 4-1* at t if the previous value from data source *4-1* at the timestamp t is *Motion begin*. A state stream A related to state a corresponds to intervals of time where the predicate $a(t)$ is valid. It is formally defined with $A = (a, \langle [t_{b_i}, t_{e_i})\rangle)$ where a is a state label and the active intervals $\langle [t_{b_i}, t_{e_i})\rangle$ a temporally ordered sequence of intervals such that $\forall i, t_{b_i}, t_{e_i} \in \mathcal{T}$, $t_{b_i} < t_{e_i} < t_{b_{i+1}}$ and $\forall t \in [t_{b_i}, t_{e_i})$, $a(t)$ is verified. The length of A, $\mathbf{len}$(A), is the sum of its active interval lengths. The size of A, noted $\#A$, corresponds to its intervals number. In Fig. 5 $\mathbf{len}(A) = 2 + 2 + 2 = 6$ and $\#A = 3$.

Operations on state streams. State streams support several useful operations. The intersection of two state streams A and B, noted $A \cap B$ is a state stream containing intervals where both A and B are active (Fig.5). Formally, $A \cap B = (a \wedge b, \langle [t_{b_i}, t_{e_i})\rangle$ such that $\forall t \in [t_{b_i}, t_{e_i}), \exists [t_{b_j}, t_{e_j}) \in A, \exists [t_{b_k}, t_{e_k}) \in B$ such that $t \in [t_{b_j}, t_{e_j})$ and $t \in [t_{b_k}, t_{e_k})$. The algorithm computing the intersection has a temporal complexity of $\Theta(Max(\#A, \#B))$. We define a temporal transformation (shifting) on a state stream B as an (α, β)-transformation. This operation produces the state stream $B^{(\alpha,\beta)} = \langle [t_{b_i} - \alpha, t_{e_i} - \beta) \mid [t_{b_i}, t_{e_i}) \in B\rangle$. Hereafter, α is called expansion and β reduction. $B^{(2,1)}$ (Fig.5). The temporal complexity of an algorithm computing a temporal transformation is $\Theta(\#B)$.

Temporal dependencies. A temporal dependency between two state streams $A \rightarrow B$ notifies that A *occurs simultaneously with* B. We call hereafter A the premise and B the conclusion. It is assessed via the length of $A \cap B$, and the

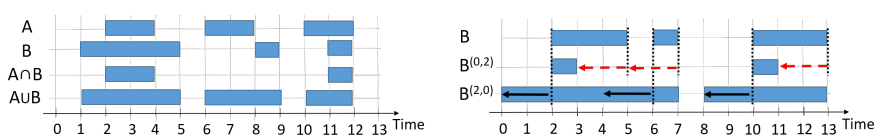


Fig. 5: Examples of intersection (left) and (α, β)-transformation (right) for state streams A and B

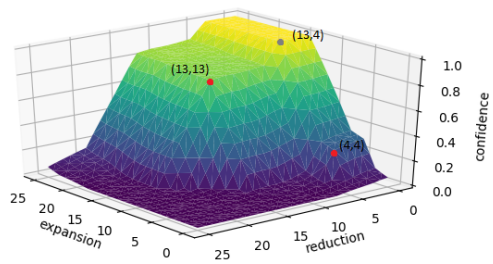


Fig. 6: Search space for temporal dependencies between 2 state streams

corresponding confidence measure [9] is:

$$\mathbf{conf}(A \rightarrow B) = \frac{\mathbf{len}(A \cap B)}{\mathbf{len}(A)}$$

It is to notice that the confidence is equal to 1 if all active intervals of A are included in the active intervals of B, i.e. A occurs always simultaneously with B. For example, in Fig 5 $\mathbf{conf}(A \rightarrow B) = 3/6$. To avoid the utilization of user-given thresholds, this confidence measure is assessed with a Pearson χ^2 test of independence. The independence hypothesis states that A and B are statistically independent and relies on the assumption that if the length of B is uniformly distributed in $\mathcal{T}$, there is no significant correlation between A and B. The given validity threshold is noted $th(len(A), len(B))$[3] and obtained for a significance level of 0.05. Simultaneity do not permit to express dependencies when it happens that B is time-delayed regarding A. To capture such relations, the statistical assessment is performed after applying (α, β)-transformations on the conclusion streams.

Problem statement Given a temporal constraint $\Delta = [min, max]$ on accepted time delays and a set of state streams $\mathcal{S}$, detect all pairwise statistically statistically significant temporal dependencies $A \rightarrow B^{(\alpha,\beta)}$ with $A, B \in \mathcal{S}$ and $\alpha, \beta \in \Delta$. The search space for a pair of streams have a size of $|\Delta|^2$ and its brute force exploration is done in $\Theta(|\Delta|^2 * Max(\#A, \#B))$. Fig.6 describes the search space for a pair of state streams with two temporal dependencies and $\Delta = [0, 25]$.

Specific dependencies selection. Due to confidence monotonicity w.r.t expansions and reductions, numerous temporal transformations can lead to valid dependencies describing the same relation. For example in Fig. 7, while $B^{(2,2)}$ and $B^{(3,1)}$ include all intervals of A and obtain a maximal confidence for a temporal dependency of type $A \rightarrow B^{(\alpha,\beta)}$, we consider that $B^{(2,2)}$ reports better the phenomenon behind this relation. To reduce the redundancy problem, one must

[3] $th(lp, lc) = \dfrac{lp * lc + \sqrt{\dfrac{3.84}{T} lp * lc(T_{obs} - lc)(T_{obs} - lp)}}{T_{obs} * lp}$

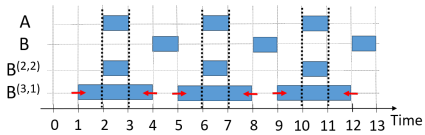


Fig. 7: Transformations leading to statistically valid dependencies

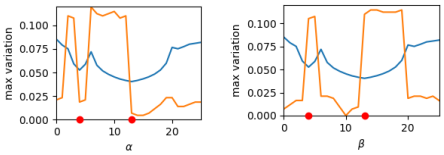


Fig. 8: Maximal gains (left) and losses (right) for Fig.6 and corresponding thresholds (blue)

select temporal transformation of B such that intervals involved in a relations intersect the most tightly their corresponding intervals in A. These transformation leads to specific temporal dependencies. One property of such a transformation is that for any of B's intervals expansion by adding (resp. subtracting) a time unit does not lead to a significant confidence gain. The opposite is true for intervals reduction (Fig. 7). As for intersection validity, we evaluate confidence variation statistically using $th(len(A \cap B^{(\alpha,\beta)}), \#B)$. However, this definition of specificity is not sufficient when several temporal relations coexist. In Fig.6 we describe 3 transformations of interest. $(13, 13)$ and $(4, 4)$ reports on real dependencies while $(13, 4)$ is an aggregation of these two temporal transformations and is more general. This is caused again by confidence monotonicity. Notice that $A \cap B^{(13,4)}$ contains the intervals of $A \cap B^{(4,4)}$ and $A \cap B^{(13,13)}$. Therefore, a specific dependency must verify the elementary confidence variation conditions (significant losses, insignificant gains) and not be included in a more general dependency.

Maximal confidence elementary variation. Specific expansion and reduction values can be extracted from different temporal relations. For example, in Fig.6, the specific expansion $\alpha = 13$ can be found for $(13, \beta)$ with $\beta \leq 20$ and specific reduction $\beta = 4$ for $(\alpha, 4)$ with $\alpha \geq 2$. We also noticed that confidence gain for (α, β) is upper-bounded by the gain obtained for (α, α). The same property stands for all elementary confidence variations: shifts (i.e (α, α)-transformations) cannot induce overlaps and removal of intervals. As a consequence, all original transitions between active and inactive stream's portions are preserved. This is not guaranteed with other transformation. Fig.8 shows maximal variations corresponding to the example described in Fig.6. Using partially the confidence variation conditions, specific values of expansions and reduction can be directly detected (red points) from maximal variations. Specific dependencies can be built using temporal order ((4,4),(13,13)) and statistically assessed via the validity test. This heuristic permits to reduce the exploration in quadratic time of the lag search space to linear time w.r.t Δ.

Algorithm 1: ITLD

```
   Data: S: set of state streams, Δ = [min, max]
   T_obs: observation duration
   Result: R : set of significant dependencies
1  R ← ∅
2  for p in S do
3      for c in S do
4          if p.label != c.label then
5              G, L, TH ← ⟨⟩
6              reference = Conf(p, c, (min, min))
7              for  α = min → max do
8                  gain ← Conf(p, c,(α + 1, α)) - reference
9                  loss ← reference - Conf(p,c,(α, α + 1))
10                 Add gain to G
11                 Add loss to L
12                 Add th(len(p) * reference, #c) to TH
13                 reference → reference + gain - loss
14             sigExp, sigRed ← GetSigVal(G, L, TH)
15             R ← R ∪ GetSpecDep(sigExp, sigRed)
16 return R
```

4 Interval Time Lag Discovery (ITLD)

This section introduces the ITLD algorithm. Given a temporal constraint $\Delta = [min, max]$, it uses the maximal confidence elementary variation property to discover specific dependencies from a set of state streams.

For each couple of state streams A and B, ITLD (lines 7 to 13) computes and stores maximal confidence elementary variations and thresholds for temporal transformations (α, α). Since elementary transformations (i.e adding/removing a unit to transformation values) do not cause interval overlaps, we infer the following equality:

$$\mathbf{conf}(A \to B^{(\alpha+1,\alpha)}) - \mathbf{conf}(A \to B^{(\alpha,\alpha)}) = \mathbf{conf}(A \to B^{(\alpha+1,\alpha+1)}) - \mathbf{conf}(A \to B^{(\alpha,\alpha+1)})$$

This relation states that adding a unit to expansion for the transformation (α, α) induce a confidence gain equal to the loss observed with subtracting a unit from expansion for $(\alpha + 1, \alpha + 1)$. This permit us to use the value of expansion gains (resp. reduction losses) to assess losses (resp. reduction gains) and vice versa. Therefore, iteration on α (lines 7-13) computes only *gain* with $(\alpha + 1, \alpha)$ and *loss* with $(\alpha, \alpha + 1)$. For the same considerations, the $(\alpha + 1, \alpha + 1)$ confidence is obtained directly with: $\mathbf{conf}(A \to B^{(\alpha+1,\alpha+1)}) = \mathbf{conf}(A \to B^{(\alpha,\alpha)}) + gain - loss$. Thus, the reference confidence needs one computation (line 6) for $\alpha = min$ and is updated on line 13 for the rest of the values. The gain/loss statistical

Procedure 2: GetSpecDep(sigExp, sigRed)

Data: sigExp, sigRed: temporally ordered specific values of expansions and reductions
Result: specDep : set of specific dependencies

```
1  specDep ← {}
2  if |sigExp| == |sigRed| then
3      i ← 0
4      while i < |sigExp| do
5          (α, β) ← (sigExp[i], sigRed[i])
6          if Conf(A → B(α, β)) > th(len(A), len(B(α, β))) then
7              Add A → B(α, β) to specDep
8          i++
9  else
10     (α, β) ← (sigExp[0], sigRed[|sigRed| − 1]
11     if Conf(A → B(α, β) > th(len(A, len(B(α, β) then
12         Add A → B(α, β) to specDep
13 return specDep
```

threshold $th(len(p \cap c^{(\alpha,\alpha)}), \#c)$ is computed in line 12. $len(p \cap c^{(\alpha,\alpha)}$ is calculated with $\mathbf{len}(p) * \mathbf{conf}(p \rightarrow c^{(\alpha,\alpha)})$ as $reference = \mathbf{conf}(p \rightarrow c^{(\alpha,\alpha)})$.

The second step of ITLD is the extraction of significant expansion and reduction values using the procedure *GetSpecVal* (line 14). This operation, done in $\Theta(|\Delta|)$, consists in verifying for each value of gain (G) and loss (L) the specificity conditions w.r.t their corresponding thresholds :

- For each element $i \in G : G[i] < TH[i]$ and $G[i-1] > TH[i-1]$
- For each element $i \in L : L[i] < TH[i]$ and $L[i-1] > TH[i-1]$

If the gain (res. loss) value at i is specific, $min + i$ is added to $sigExp$ (res. $sigRed$). The first value in L and last in G are tested with a relaxed version of the higher conditions to capture dependencies with potential specific values that are out of Δ:

- $L[0] > TH[0]$ means that the specific loss $l \leq min \Rightarrow min$ is added to $sigExp$
- $G[len(G) - 1] > TH[0]$ means that the specific gain $l \geq max \Rightarrow max$ is added to $sigExp$

For example, in Fig. 8.a whith $\alpha \in \Delta = [0, 10]$, the real specific value 14 is out of bounds of Δ but statistically significant confidence variations are detected: max is considered as a specific gain to maintain this information.

The last step *GetSpecDep* (Procedure 2) builds specific dependencies candidates using temporal order (first specific expansion with first specific reduction etc.) and tests if they are statistically valid (line 2 to 8). If an unequal number of specific expansion and reductions is found, the most general dependency (with the greatest expansion and lowest reduction, line 10 to 12) is considered and tested permitting to avoid losing temporal information.

5 Experiments

In the section, we describe experiments on both simulated and real world data. We also compare qualitatively and quantitatively ITLD with the existing similar approaches TEDDY [9] and PIVOTMiner [4] that we adapted to state streams. Algorithms were implemented in Python[4] (with DBSCAN from scikit-learn for PIVOTMiner) and tested on a Core i7 2.1Ghz with 8GB memory running Windows 10.

5.1 Results on synthetic data

We developed a motion simulation tool[4] in order to obtain data sets using custom scenarios (trajectories, activity density, speed ...). This controlled testbed generates data sets with a corresponding ground truth permitting to evaluate the accuracy of discovered temporal dependencies. In the following, we use the F1-Score, i.e the harmonic mean of precision and recall. Several matching condition types are used. Given two dependencies $D_1 = A_1 \rightarrow B_1^{(\alpha_1,\beta_1)}$ and $D_2 = A_2 \rightarrow B_2^{(\alpha_2,\beta_2)}$:

- Qualitative matching: D_1 and D_2 are equivalent if $A_1 = A_2$ and $B_1 = B_2$.
- Exact matching: D_1 and D_2 are equivalent if they match qualitatively and $(\alpha_1, \beta_1) = (\alpha_2, \beta_2)$
- Relaxed θ Matching: for $\theta \geq 0$, D_1 and D_2 are equivalent if they match qualitatively and $|\alpha_1 - \alpha_2| \leq \theta$ and $|\beta_1 - \beta_2| \leq \theta$.

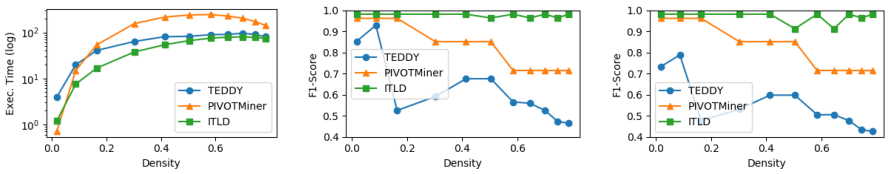
In order to keep this comparison fair, we included the Δ constraint to PIVOTMiner by simply keeping interval vectors with time gaps in Δ for the clustering step. In the case of a pair of interval streams with several temporal dependencies (e.g Fig.6), the results given by TEDDY corresponds to the general temporal relation ($(13, 4)$ in Fig.6) that matches qualitatively but not with respect to the temporal information. Moreover, as TEDDY considers temporal transformation (α, β) with $\alpha \geq \beta$, it outputs multiple dependencies if $\alpha < \beta$. For example, for $A \rightarrow B^{(9,10)}$, two results with $(9, 9)$ and $(10, 10)$ are given. In this case, we consider one dependency as a true positive and others as false positives.

Robustness to density Activity within a sensed environment can be more or less intense. In order to be efficient, a pattern mining approach must be able to give precise results for both sparse and dense data. We defined a linear trajectory with equidistant sensors and ran 11 simulations varying the number of occurrences for the same duration T_{obs} and object speed (cf Table 1). The intervals number increases with the number of event occurrences for sparse data (100 to 5000 occurrences) and decreases for high density event occurrences due to intervals overlap. The streams active length always increases when the event occurrences increases and have a density $(\frac{len(stream)}{T_{obs}})$ ranging from 1.7% to 78.5%.

[4] https://github.com/AElOuassouli/Quantitaive-Interval-Stream-Mining

Table 1: 11 simulated data sets with increasing occurrences number (*Occ*). *#Int*: average number of intervals per stream, *Den*: average density % of T_{obs}

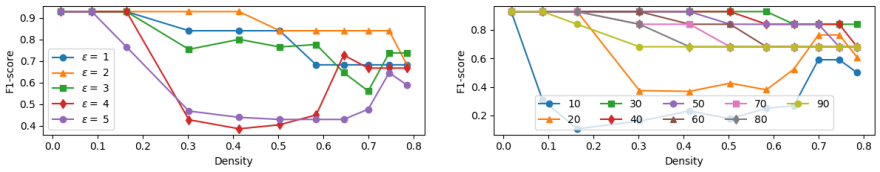
Occ	#Int	Den	Occ	#Int	Den	Occ	#Int	Den	Occ	#Int	Den
100	95	1.7	2000	1275	30.3	5000	1616	58.6	8000	1386	74.6
500	452	8.7	3000	1509	41.5	6000	1570	64.7	9000	1263	78.5
1000	798	16.4	4000	1606	50.5	7000	1493	69.9			



(a) Execution Time (b) Qualitative Matching (c) Exact Matching

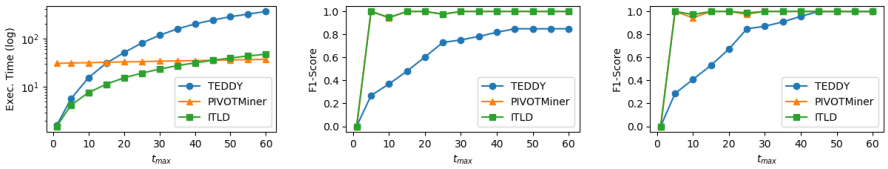
Fig. 9: Execution time and F1-Scores w.r.t to density of streams

We report in Fig. 10 results for ITLD, TEDDY and PIVOTMiner ($\epsilon = 1$, *min-sup*$= 2/3$) with $\Delta = [0, 14]$, $T_{obs} = 10000$. Fig. 10a shows that ITLD runs faster in comparison with PIVOTMiner and TEDDY. TEDDY benefits from its early pruning criteria for non-significant dependencies but requires more processing time for significant dependencies. PIVOTMiner suffers from its quadratic build of delta vectors linking each interval with all others. Therefore, its running time increases dramatically with intervals number. Notice that PIVOTMiner can be improved significantly using a sliding window technique. F1-Scores reported in Fig. 9b and Fig. 9c shows that ITLD is robust to density even for exact matching, in comparison with PIVOTMiner and TEDDY. The latter is able to detect significant dependencies (high recalls) but outputs a significant amount of false positives impacting precision. One observation is that these false positives often have large temporal transformations causing a significant increase in intervals lengths and, by extension, intersection. Large temporal transformations deforms, in some extent, the original temporal information and can lead to false statistically valid dependencies. This problem is tackled by ITLD as it considers confidence variations for non-deformed intervals. With the specified parameters, PIVOTMiner behaves well for sparse data and gives similar results to ITLD but lower recalls are obtained for dense data. This is explained by the fact that overlaps caused by great densities causes a fall in dependencies counts and then requires to lower the minimum support which may cause a pattern flooding problem. We conclude that the usage of intersection is more robust to variations of densities: both TEDDY and ITLD obtain good recalls. For a fair comparison, we executed PIVOTMiner varying ϵ and minimum support. The results are reported in Fig. 9. Even for the best ϵ value ($\epsilon = 2$), PIVOTMiner is sensitive to density. Low minimum supports causes pattern flooding (10%, 20%) and the best qualitative results are given for 30%. These results shows



(a) ϵ variation. *min-supp* $= 2/3$ (b) *min-supp* variation. $\epsilon = 1$

Fig. 10: F1 scores (qualitative matching) for PIVOTMiner



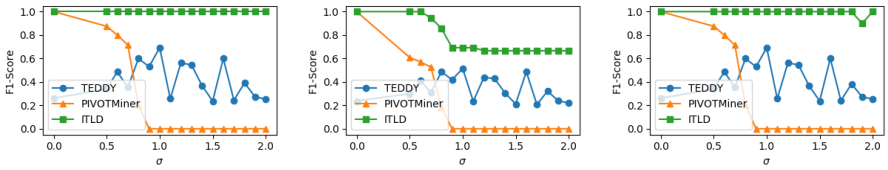
(a) Execution Time (b) Exact Matching (c) Relaxed Matching $\theta = 1$

Fig. 11: Execution time and F1-Scores w.r.t to t_{max} ($t_{min} = 0$)

that using automatic statistical tests permits to adapt the significance threshold to activity intensity, contrary to user-given significance thresholds approaches requiring continuous parameter tuning.

Influence of $|\Delta|$ The empirical study of the influence of Δ was done on a data set generated using a linear trajectory with 10 equidistant steps. Its average density is 16% (sparse data) and contains 1000 occurrences in a $T_{obs} = 10000$. The three algorithms were executed with different $\Delta = [0, t_{max}]$, with $t_{max} \in [0, 60]$. ITLD runs faster than TEDDY (up to 7 times for $t_{max} = 60$) thanks to its linear complexity w.r.t Δ. As expected, running time of PIVOTMiner is marginally affected by Δ as this constraint is simply used to reduce the number of vectors for the clustering step. As this data set is sparse and noiseless, both PIVOTMiner and ITLD obtain maximum F1-Scores for the reasons discussed higher. TEDDY obtains higher recalls and a growing precision with t_{max} for this data set. That is explained by the number of true positives that grows with t_{max} due to the simulation scenario. TEDDY is capable of finding true positives but fails somehow to detect true negatives. TEDDY is designed to extract correlations based on intersection length and do not integrate directly the succession information. ITLD benefits from the intersection model assessment and, in the same time, takes into account directly temporal successions as it uses confidence elementary variations that can be seen as the number of impacted intervals.

Robustness to Temporal variability In real world context, a same phenomenon can occur with slight temporal variation. For example, two pedestrians can perform the same trajectory with slightly different velocities. In our motion



(a) Qualitative matching (b) Exact matching (c) Relaxed matching $\theta = 2$

Fig. 12: F1 scores over σ

Table 2: Data set corresponding to the experiment described in Fig.2

Name	#Int	Len	Name	#Int	Len	Name	#Int	Len	Name	#Int	Len
1-1	1403	4303	2-1	8640	47881	3-1	3909	14644	4-1	9686	30257
1-2	851	3257	2-2	2099	4947	3-2	3699	13423	4-2	4578	13397
			2-3	8548	26847				4-3	9825	21273

simulation tool, for a typical speed v each simulation instance have a velocity $V \sim \mathcal{N}(v, \sigma^2)$ with σ^2 representing the variance of the distribution (inspired by [13]). Fig. 12 reports on F1-score obtained on data sets with $v = 10$ and $\sigma \in [0, 2]$ ($T_{obs} = 10000$, avg. density $= 25\%$) and $\Delta = [0, 15]$, $\epsilon = 1$, *min-supp*$= 2/3$. The resulting F1-Scores show that ITLD is the most robust to temporal variability. Notice that even if the F1-Score with exact matching decreases with σ, the discovered dependencies remains close to expected results: the F1-Score is almost maximal using the relaxed matching ($\theta = 2$). We noticed that greater values of ϵ provides better results for PIVOTMiner: as the speeds follows a normal distribution centered in v, increasing the maximal distance between interval points captures more results. The same thing can be noticed with lowering the minimum support : there are less occurrences with the exact value of v. In real world scenarios, state streams often describe several temporal phenomena with different temporal variability and density characteristics: choosing a optimal parameter for one phenomenon may not fit for the others.

5.2 Results on real-world data

We experimented our algorithm using real-world sensor data gathered from the motion sensor system described in Fig.2. This data set (Table 2) describes motion activity in the office area during 18 working days between 6 am and 8 pm. First observations showed that the data set contains a significant amount of noise (e.g detection of shadows, sudden luminosity changes) and several omissions were observed (e.g when a car passes through an analysis zone with a great speed). To assess the quality of the results, a qualitative "approximate" ground truth was built based on our ground observations and the manual analysis of the resulting streams. We basically eliminated impossible dependencies w.r.t to the time lag

Table 3: Results on the FOX data set for $\Delta = [0, 40]$ (Qualitative matching)

	#Res	Precision	Recall	F1-Score	Accuracy	Run. time (s)
PIVOTMiner	15	**1**	0.25	0.43	0.59	1704
TEDDY	94	0.54	**0.98**	0.69	0.51	433
ITLD	64	**0.81**	**0.92**	**0.86**	**0.83**	**170**

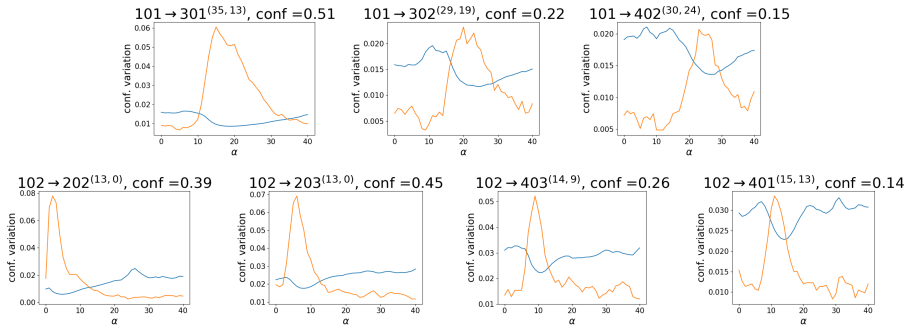


Fig. 13: Expansion confidence variation (in orange) and statistical threshold (blue) for results given by ITLD

restriction and added observed "unusual" behaviors (e.g activation of the state *4-1* then *4-2* and *3-2* is forbidden but is actually often observed).

We ran ITLD, TEDDY and PIVOTMiner (*min-support*= 0.1 and $\epsilon = 1$) with $\Delta = [0, 40]$. Table 3 describes the obtained results based on a qualitative matching. ITLD completed the discovery process faster in comparison with the other algorithms (2.5 time faster than TEDDY and 10 times faster than PIVOTMiner). The values of precision, recall and F1-scores permits to validate our observation on synthetic data. Indeed, TEDDY obtain a good recall but suffers from the amount of false positives impacting its precision. In the other hand, PIVOTMiner is precise but have a weak recall with these parameters. ITLD is characterized by results with a good recall/precision performance and without using user-given parameters (excepting Δ). The analysis of the errors shows that their majority are false positives due to low statistical thresholds and to the large observation duration in comparison with streams length (average density of 2%). Fig. 13 describes several results given by ITLD and their corresponding expansion confidence variation. One can observe that the 'characteristic form' of significant dependencies is well detected by the dynamic statistical threshold. Moreover, the temporal information remains consistent with the environment configuration (cf. Fig.2).

6 Conclusion and future work

Data streams gathered from sensor systems are challenging to temporal knowledge mining approaches. The current work investigates the problem of mining

time delayed relations between sensor interval-based state streams. While most of the existing approaches focus on the problem of finding frequent patterns, our work is devised to identify quantitative delayed dependencies between interval streams on the basis of the automatic statistical test of their intersection length as introduced by [9]. First, we extended this dependency model by proposing a new significance definition based on the analysis of confidences elementary variations. In addition to reducing the exploration of a quadratic search space to a linear operation, this permits to better capture the time delayed transitions between streams while benefiting from the intersection assessment. Second, we designed the ITLD algorithms that uses the later property. We also performed experiments on both synthetic and real-world data, comparing the capabilities of our approach with available methods. The results showed that ITLD is more robust to density of streams and temporal variability while not requiring a user-given significance parameter. Moreover, our real-life experiment showed that it is possible to include data given by video analysis approaches in a temporal knowledge discovery process.

Our work can be extended in various directions. First, our approach may permit to define a non-ambiguous delayed relations. Indeed, the confidence variation characteristic form can be used to obtain typical intervals duration in addition to boundaries time lags. Second, pairwise dependencies can be used to build larger "patterns" involving multiple state streams.

References

1. Allen, J., Hayes, P.: A Common-Sense Theory of Time. In: IJCAI Int. Jt. Conf. Artif. Intell. pp. 528–531. IJCAI'85, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (1985)
2. Casas-Garriga, G.: Discovering unbounded episodes in sequential data. In: Eur. Conf. Princ. Data Min. Knowl. Discov. pp. 83–94 (2003)
3. Guyet, T., Quiniou, R.: Extracting temporal patterns from interval-based sequences. IJCAI Int. Jt. Conf. Artif. Intell. pp. 1306–1311 (2011)
4. Hassani, M., Lu, Y., Wischnewsky, J., Seidl, T.: A geometric approach for mining sequential patterns in interval-based data streams. In: Fuzzy Syst. (FUZZ-IEEE), 2016 IEEE Int. Conf. pp. 2128–2135 (2016)
5. Höppner, F., Peter, S.: Temporal interval pattern languages to characterize time flow. Wiley Interdiscip. Rev. Data Min. Knowl. Discov. **4**(3), 196–212 (2014)
6. Mannila, H., Toivonen, H., Inkeri Verkamo, A.: Discovery of Frequent Episodes in Event Sequences. Data Min. Knowl. Discov. **1**(3), 259–289 (jan 1997)
7. Nakagaito, F., Ozaki, T., Ohkawa, T.: Discovery of quantitative sequential patterns from event sequences. In: 2009 IEEE Int. Conf. Data Min. Work. pp. 31–36 (2009)
8. Pei, J., Han, J., Mortazavi-asl, B., Pinto, H., Chen, Q., Dayal, U., Hsu, M.c.: PrefixSpan: Mining Sequential Patterns Efficiently by Prefix-Projected Pattern Growth. pp. 215–224 (2001)
9. Plantevit, M., Robardet, C., Scuturici, V.M.: Graph Dependency Construction Based on Interval-based Event Dependencies Detection in Data Streams. Intell. Data Anal. **20(2)**, 223–256 (mar 2016)

10. Ruan, G., Zhang, H., Plale, B.: Parallel and quantitative sequential pattern mining for large-scale interval-based temporal data. In: Big Data (Big Data), 2014 IEEE Int. Conf. pp. 32–39 (2014)
11. Tang, L., Li, T., Shwartz, L.: Discovering lag intervals for temporal dependencies. In: Proc. 18th ACM SIGKDD Int. Conf. Knowl. Discov. data Min. p. 633 (2012)
12. Tao Li, Sheng Ma: Mining Temporal Patterns Without Predefined Time Windows. In: Fourth IEEE Int. Conf. Data Min. pp. 451–454. IEEE (2004)
13. Wang, W., Zeng, C., Li, T.: Discovering multiple time lags of temporal dependencies from fluctuating events. In: Asia-Pacific Web Web-Age Inf. Manag. Jt. Int. Conf. Web Big Data. pp. 121–137 (2018)